

ДЗ №2. Онлайн-курсы

1. Дополнение схемы

Добавлены две таблицы: payments и lesson_progress.

payments хранит информацию об оплате записи на курс. Статус оплаты: pending, paid или failed.

lesson_progress хранит прохождение конкретных уроков студентом. Поле completed показывает, завершён урок или нет.

Связь User и Course является M:N и реализована через Enrollment. Так можно хранить дату записи, цену покупки и статус обучения, а составной ключ не даёт создать повторную запись на тот же курс.

Если выполнить DROP TABLE courses CASCADE, таблица courses будет удалена, а также будут удалены внешние ключи, которые от неё зависят. DROP SCHEMA hw_edtech CASCADE удаляет всю учебную схему вместе с её объектами.

Индексы

- idx_courses_teacher - поиск курсов преподавателя.
- idx_enrollments_course - поиск записей на конкретный курс.
- PK lessons(course_id, lesson_no) - поиск уроков курса по порядку.
- idx_reviews_course - поиск отзывов курса.
- idx_payments_course и idx_progress_lesson - запросы к новым таблицам.

SQL дополнения схемы

– Сначала выполнить ДЗ_1.sql.

BEGIN;

SET search_path TO hw_edtech;

```
CREATE TABLE IF NOT EXISTS payments (  
  payment_id SERIAL PRIMARY KEY,  
  user_id INTEGER NOT NULL,  
  course_id INTEGER NOT NULL,  
  status VARCHAR(20) NOT NULL DEFAULT 'pending'  
  CHECK (status IN ('pending', 'paid', 'failed')),  
  paid_at TIMESTAMP,  
  UNIQUE (user_id, course_id),  
  FOREIGN KEY (user_id, course_id)  
  REFERENCES enrollments(user_id, course_id)  
);
```

```
CREATE TABLE IF NOT EXISTS lesson_progress (  
  progress_id SERIAL PRIMARY KEY,  
  user_id INTEGER NOT NULL,  
  course_id INTEGER NOT NULL,  
  lesson_no INTEGER NOT NULL,  
  completed BOOLEAN NOT NULL DEFAULT FALSE,
```

```

    UNIQUE (user_id, course_id, lesson_no),
    FOREIGN KEY (user_id, course_id)
        REFERENCES enrollments(user_id, course_id),
    FOREIGN KEY (course_id, lesson_no)
        REFERENCES lessons(course_id, lesson_no)
);

CREATE INDEX IF NOT EXISTS idx_payments_course
ON payments(course_id);

CREATE INDEX IF NOT EXISTS idx_progress_lesson
ON lesson_progress(course_id, lesson_no);

COMMIT;

```

2. Демонстрация нарушений

Скрипт создаёт тестовые записи, проверяет пять ограничений и в конце делает ROLLBACK, поэтому тестовые данные не остаются в базе.

– Сначала выполнить ДЗ_1.sql и ДЗ_2_схема.sql.

```

BEGIN;
SET LOCAL search_path TO hw_edtech;
SET LOCAL client_min_messages TO NOTICE;

INSERT INTO users (user_id, full_name, email) VALUES
    (-1, 'Анна', 'teacher@example.com'),
    (-2, 'Олег', 'student@example.com');

INSERT INTO courses (course_id, teacher_id, title, price, teacher_rate)
VALUES (-1, -1, 'Основы SQL', 3000, 0.7);

INSERT INTO lessons (course_id, lesson_no, title)
VALUES (-1, 1, 'Введение');

INSERT INTO enrollments (user_id, course_id, price_paid)
VALUES (-2, -1, 3000);

-- 1. CHECK
DO $$
BEGIN
    INSERT INTO reviews (user_id, course_id, rating)
    VALUES (-2, -1, 6);
EXCEPTION
    WHEN check_violation THEN
        RAISE NOTICE 'Оценка курса должна быть от 1 до 5. Текст ошибки: %', SQLERRM;
END;
$$;

-- 2. FOREIGN KEY
DO $$

```

```
BEGIN
  INSERT INTO lessons (course_id, lesson_no, title)
  VALUES (-999, 1, 'Введение');
EXCEPTION
  WHEN foreign_key_violation THEN
    RAISE NOTICE 'Нельзя добавить урок к несуществующему курсу. Текст ошибки: %', SQLERRM;
END;
$$;

-- 3. UNIQUE
DO $$
BEGIN
  INSERT INTO users (full_name, email)
  VALUES ('Иван', 'student@example.com');
EXCEPTION
  WHEN unique_violation THEN
    RAISE NOTICE 'Пользователь с такой почтой уже зарегистрирован. Текст ошибки: %', SQLERRM;
END;
$$;

-- 4. NOT NULL
DO $$
BEGIN
  INSERT INTO users (full_name, email)
  VALUES (NULL, 'new@example.com');
EXCEPTION
  WHEN not_null_violation THEN
    RAISE NOTICE 'При регистрации нужно указать имя. Текст ошибки: %', SQLERRM;
END;
$$;

-- 5. PRIMARY KEY
DO $$
BEGIN
  INSERT INTO lessons (course_id, lesson_no, title)
  VALUES (-1, 1, 'Повтор');
EXCEPTION
  WHEN unique_violation THEN
    RAISE NOTICE 'В курсе уже есть урок с таким номером. Текст ошибки: %', SQLERRM;
END;
$$;

ROLLBACK;
```

3. Документирование нарушений

№	Ограничение	Выполняемый запрос (SQL)	Сообщение СУБД	Понятное сообщение	Как исправить
1	CHECK	INSERT INTO reviews (...) VALUES (-2, -1, 6);	new row for relation "reviews" violates check constraint "reviews_rating_check"	Оценка должна быть от 1 до 5.	Указать оценку 1-5.
2	FOREIGN KEY	INSERT INTO lessons (...) VALUES (-999, 1, 'Введение');	insert or update on table "lessons" violates foreign key constraint "lessons_course_id_fkey"	Такого курса нет.	Указать существующий course_id.
3	UNIQUE	INSERT INTO users (...) VALUES ('Иван', 'student@example.com');	duplicate key value violates unique constraint "users_email_key"	Такая почта уже используется.	Указать другую почту.
4	NOT NULL	INSERT INTO users (...) VALUES (NULL, 'new@example.com');	null value in column "full_name" of relation "users" violates not- null constraint	Нужно указать имя.	Заполнить full_name.
5	PRIMARY KEY	INSERT INTO lessons (...) VALUES (-1, 1, 'Повтор');	duplicate key value violates unique constraint "lessons_pkey"	Урок с таким номером уже есть.	Выбрать другой номер урока.

В результате проверяются CHECK, FOREIGN KEY, UNIQUE, NOT NULL и PRIMARY KEY. Все тестовые вставки отменяются командой ROLLBACK.